

PASTA Threat Model Analysis

Sneaker Company Mobile App

Generated: September 29, 2026 • Process for Attack Simulation and Threat Analysis

Executive Summary

This document presents a comprehensive threat model of the Sneaker Company's new mobile application using the PASTA framework. The analysis covers all seven stages of the threat modeling process to identify security vulnerabilities and recommend risk mitigation strategies. Key findings indicate that SQL injection and weak encryption represent the highest risks, requiring immediate implementation of security controls before product launch.

Stage I: Business Objectives Definition

Objective: Understand the business purpose and expected functionality of the application.

Key Business Objectives:

- 1. Seamless seller-shopper connection:** The application should facilitate easy user signup, login, and account management with secure communication channels between buyers and sellers.
- 2. Data privacy and user trust:** Users must have confidence that their personal information is being handled responsibly and securely, as this is a primary business concern.
- 3. Secure payment processing:** Multiple payment options should be available with clear, quick transactions while maintaining legal compliance and avoiding financial/security liabilities.

Stage II: Technological Scope

Objective: Define the application architecture and identify key technologies requiring security evaluation.

Technologies Evaluated:

- **API (Application Programming Interface):** Third-party APIs for added functionality
- **PKI (Public Key Infrastructure):** AES (symmetric) and RSA (asymmetric) encryption
- **SHA-256:** Hash function for protecting passwords and sensitive data
- **SQL Database:** Storage and retrieval of product listings, seller information, and user data

Priority Technology: SQL Database

SQL database operations are prioritized for security evaluation because they directly handle the most sensitive user data including personal information, payment details, and transaction history. Database vulnerabilities create the largest attack surface for unauthorized data access and manipulation. A successful SQL injection attack could compromise all customer data, violate payment card industry (PCI) compliance, and result in significant legal and financial consequences.

Stage III: Data Flow Analysis

Objective: Analyze how information flows through application processes and identify data handling procedures.

Critical Data Flow Processes:

- User authentication (login/password validation via SQL)
- Product database queries (search functionality with user input)
- Payment processing (credit card data encryption and transmission)
- Messaging system (communication between buyers and sellers)
- User data storage (personal information, addresses, payment methods)
- Rating and review submission (user-generated content storage)

Stage IV: Threat Analysis

Objective: Identify potential threats that could exploit the technologies and data flows identified in previous stages.

Identified Threats:

1. SQL Injection Attacks: Attackers could exploit unvalidated user input in search fields, login forms, and message fields to execute arbitrary SQL commands. This could allow unauthorized access to user credentials, payment information, and personal data stored in the database.

2. Malware & Virus Distribution: Threat actors could inject malicious code through the messaging system or compromised third-party APIs to steal user credentials, payment data, or hijack user sessions. Malware could also target the server infrastructure to gain unauthorized database access.

3. Man-in-the-Middle (MITM) Attacks: If communications are not properly encrypted (TLS), attackers could intercept payment data, authentication tokens, or user information transmitted between the app and servers.

4. Brute Force Authentication Attacks: Attackers could attempt to guess user passwords through automated login attempts, potentially compromising user accounts and accessing personal/payment information.

Stage V: Vulnerability Analysis

Objective: Identify specific weaknesses in the application that could be exploited by threat actors.

Identified Vulnerabilities:

1. Unvalidated SQL Queries (CWE-89): If user input from search fields, usernames, or payment forms is not sanitized before being passed to SQL queries, attackers can inject malicious SQL code. Example: Entering ' OR '1'='1 in search fields could bypass query logic and return unauthorized data.

2. Weak Encryption Implementation: If AES or RSA encryption is improperly configured, uses insufficient key sizes (less than 256-bit for AES), or has flawed key management, encrypted payment and user data could be decrypted by sophisticated attackers. This violates PCI-DSS compliance requirements.

3. Missing Input Validation (CWE-20): Without comprehensive input validation on all user-controlled fields, multiple injection attacks (SQL injection, XSS, command injection) become possible.

4. Insufficient API Authentication: Third-party APIs lacking proper authentication mechanisms (API keys, OAuth) could be exploited to access, modify, or delete application data.

Stage VI: Attack Tree Development

Objective: Create a visual map of how threat actors could potentially exploit vulnerabilities to achieve their goals.

Primary Attack Goal: Compromise User & Payment Data

Attack Paths:

- **Path 1** - SQL Injection → Login Bypass → Account Access → Payment Data Theft
- **Path 2** - SQL Injection → Search Field → Database Dump → Credential Harvest
- **Path 3** - Weak Encryption → Intercept Encrypted Data → Decrypt → Payment Fraud
- **Path 4** - Brute Force Authentication → Account Takeover → Payment Methods Access
- **Path 5** - MITM Attack → Intercept API Calls → Steal Session Tokens → Unauthorized Actions

Stage VII: Risk Mitigation & Security Controls

Objective: Implement defenses and safeguards to reduce the likelihood and impact of identified threats and vulnerabilities.

Recommended Security Controls:

1. Input Validation & Parameterized Queries: Implement strict input validation for all user inputs using whitelisting. Use parameterized SQL queries (prepared statements) exclusively to prevent SQL injection. Validate data type, length, and format on both client and server side.

2. Multi-Factor Authentication (MFA): Require MFA (e.g., TOTP, SMS) for all user accounts, especially for seller accounts with transaction access. This significantly reduces the success rate of brute force and credential compromise attacks.

3. End-to-End Encryption: Implement AES-256 encryption for all sensitive data at rest. Use TLS 1.3 with strong cipher suites for all API communications. Ensure proper key management and regular key rotation policies.

4. Rate Limiting & Account Lockout: Configure rate limiting on authentication endpoints to prevent brute force attacks. Implement account lockout after 5 failed login attempts (30-minute lockout). Add CAPTCHA after 3 failed attempts.

5. Web Application Firewall (WAF): Deploy WAF (e.g., ModSecurity, AWS WAF) to detect and block malicious traffic patterns, SQL injection attempts, XSS payloads, and known attack signatures before reaching the application.

6. Security Logging & Monitoring: Implement comprehensive logging of all database queries, authentication attempts, API calls, and payment transactions. Use SIEM tools (e.g., Splunk, ELK) for real-time threat detection and automated incident response.

Implementation Roadmap

Priority	Control	Effort	Timeline
Critical	Input Validation & Parameterized Queries	High	Week 1-2
Critical	End-to-End Encryption	High	Week 2-3
High	Multi-Factor Authentication	Medium	Week 3-4
High	WAF Deployment	Medium	Week 4
High	Rate Limiting & Account Lockout	Low	Week 4
Medium	Security Logging & Monitoring	High	Week 5-6

Launch Readiness Checklist

- ✓ Input validation deployed on all user input fields
- ✓ Parameterized SQL queries implemented throughout application
- ✓ AES-256 encryption enabled for sensitive data at rest
- ✓ TLS 1.3 configured for all API communications
- ✓ Multi-factor authentication enabled for all accounts
- ✓ Rate limiting and account lockout mechanisms active
- ✓ Web Application Firewall deployed and configured
- ✓ Security logging and monitoring operational
- ✓ Penetration testing completed (recommend third-party)
- ✓ Security training completed for development and operations teams

Recommendation: Proceed with product launch only after all critical and high-priority controls are implemented and verified through security testing.

References & Resources

PASTA Framework:

- PASTA Whitepaper: <https://pasta.owasp.org/>
- OWASP Threat Modeling: https://owasp.org/www-community/Threat_Model

Security Standards:

- OWASP Top 10 2021: <https://owasp.org/www-project-top-ten/>
- NIST Cybersecurity Framework: <https://www.nist.gov/cyberframework>
- PCI-DSS Compliance: <https://www.pcisecuritystandards.org/>

Vulnerability References:

- CWE-89 (SQL Injection): <https://cwe.mitre.org/data/definitions/89.html>
- CWE-20 (Improper Input Validation): <https://cwe.mitre.org/data/definitions/20.html>
- CWE-79 (Cross-site Scripting): <https://cwe.mitre.org/data/definitions/79.html>